There are a lot of other Ruby methods that give you insight into the internals of how code runs that you can reveal via p and puts. Most of them are a little arcane for our purposes, but Aaron Patterson has a great rundown.[5]

Git Bisect

git bisect is the kind of tool you'll use about once every six months, but when you use it, you'll be totally thrilled that it exists.

git bisect is indicated when something has gone wrong in your code, and you believe it to be the result of a code change but you cannot isolate which change resulted in the problem. git bisect's goal is to isolate the commit where the change occurred.

Prescription 42	Use git bisect to track down mysterious failures in your code when you have no idea how they were inserted.

You start using git bisect with two commands:

```
$ git bisect start
$ git bisect bad
```

The first command puts Git into what I'll call *bisect mode*, and the second command says that the current Git snapshot is bad, meaning it contains the behavior you're trying to fix.

You then switch the Git snapshot to the secure hash algorithm of a previous commit that you believe is good because it doesn't have the behavior. And you tell Git that branch is good. For example, I'd use this if I've determined that SHA 34ace43 is good:

```
$ git checkout 34ace43
$ git bisect good
```

You can combine those into one command, git bisect good 34ace43.

Git now does something really neat. It derives a straight line of commits between the good one and the bad one, picks the middle of that line, and checks itself into that commit. You'll see some commentary on the console explaining how many commits are in the line and roughly how many steps Git expects the bisect to take.

Your job is to do whatever you need to in the newly entered commit to determine if it is "good" or "bad" based on whether the incorrect behavior exists.

5. https://tenderlovemaking.com/2016/02/05/i-am-a-puts-debuggerer.html